



Instituto Superior de
Engenharia do Porto

RELATÓRIO

ALGAV

Grupo 79

Miguel Silva | 1111180

Carlos Paula | 1212032

Gonçalo Silva | 1221719

Índice

Introdução	3
Contexto e Motivação	3
User Stories	4
Breve explicação do código	4
US6.3.2 – Estudo de complexidade	5
US6.3.3 - Heurísticas	7
US6.3.1 – Incluir todo o staff	9
Conclusão.....	14

Introdução

O presente trabalho, realizado no âmbito da Sprint 2 da unidade curricular de ALGAV do curso de Engenharia Informática do ISEP, tem como objetivo desenvolver e implementar soluções eficientes para o problema de agendamento de cirurgias em salas de operação hospitalares. A partir de casos de uso específicos, o sistema visa otimizar o planeamento das cirurgias, garantindo a utilização eficiente dos recursos humanos e materiais disponíveis.

Contexto e Motivação

O agendamento de cirurgias é um problema complexo que envolve a coordenação de múltiplos membros de uma equipa, onde estão incluindo médicos, anestesiistas, enfermeiros e assistentes, além de respeitar as limitações de tempo e recursos das salas de operação. A necessidade de minimizar atrasos, evitar conflitos de agendas e distribuir a carga de trabalho de forma balanceada são desafios centrais enfrentados por sistemas de planeamento hospitalar.

Neste Sprint 2 o desafio é desenvolver as seguintes US:

Adaptação do sistema para incluir todas as fases de uma cirurgia e o staff necessário para sua execução (6.3.1).

Estudo de complexidade do problema para determinar até que ponto é viável encontrar a solução ótima (6.3.2).

Desenvolvimento de heurísticas que proporcionem soluções boas e utilizáveis em tempo útil (6.3.3).

User Stories

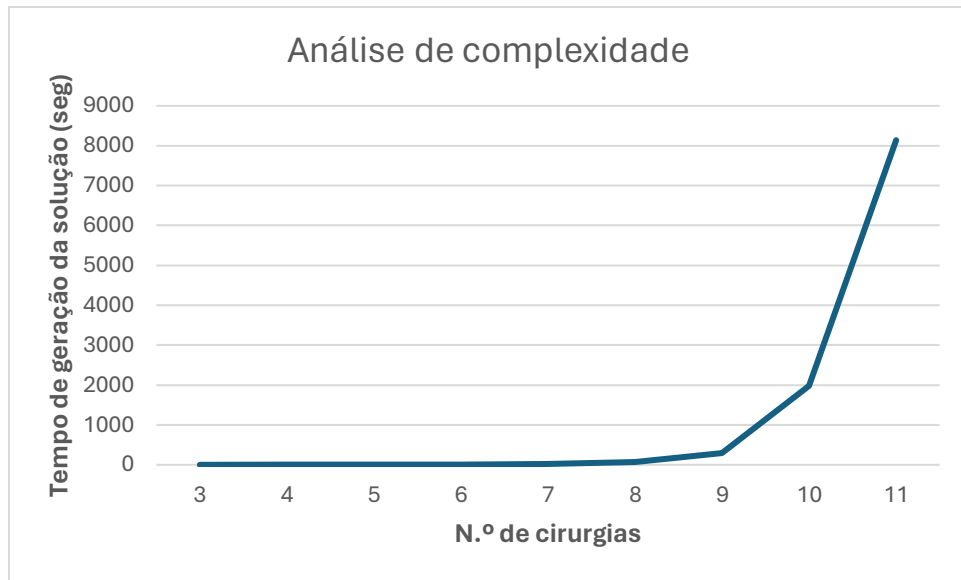
Breve explicação do código

Método	Explicação
obtain_better_sol	Função principal para obter a melhor solução possível. Inicializa o tempo (get_time), chama obtain_better_sol1 para gerar soluções e retorna a melhor solução (better_sol).
obtain_better_sol1	Gera permutações de todas as cirurgias (permutation), recalcula agendas e tenta agendar cirurgias. Após cada tentativa, chama update_better_sol para atualizar a solução caso seja melhor.
update_better_sol	Verifica se a solução atual é melhor que a anterior, baseada no horário de fim da última cirurgia. Se for melhor, atualiza o better_sol.
evaluate_final_time	Avalia o horário de fim de uma solução baseada na última cirurgia agendada na sala.
schedule_all_surgeries	Realiza o agendamento das cirurgias numa sala para o dia especificado. Limpa dados antigos, calcula disponibilidade, ordena as cirurgias por duração e tenta agendá-las todas (availability_all_surgeries).
availability_all_surgeries	Agenda uma lista de cirurgias. Para cada cirurgia, verifica a disponibilidade e agenda a cirurgia na sala e nas agendas dos médicos envolvidos.
availability_operation	Determina os horários disponíveis para agendar uma cirurgia específica. Calcula a interseção das agendas dos médicos e da sala para encontrar intervalos disponíveis.
free_agenda0	Calculam os intervalos de tempo livres para uma lista de horários ocupados.
adapt_timetable	Ajusta os horários livres ao horário de trabalho do staff (timetable).
intersect_all_agendas	Calcula os intervalos de tempo em comum entre as agendas dos médicos e da sala.
insert_agenda	Insere um novo intervalo de horário numa agenda, garantindo que a agenda continua ordenada.
insert_agenda_doctors	Insere o horário agendado nas agendas dos médicos envolvidos na cirurgia.
remove_unf_intervals	Remove intervalos de tempo que são menores que o necessário para realizar a cirurgia.

US6.3.2 – Estudo de complexidade

As an Admin, I want to know till what dimension in terms of number of surgeries is possible to ask for the better solution.

N. of Surgeries	N. of solutions	Best Schedule of activities (including surgeries) of the operation room	Final Time for the last Surgery (minutes)	Time to generate the solution (s)
3	6	[(480, 539, so100001), (540, 629, so100002), (630, 704, so100003)]	704	0,0119
4	24	[(480, 539, so100001), (540, 614, so100003), (615, 674, so100004), (675, 764, so100002)]	764	0,1567
5	120	[(500, 559, so100004), (560, 634, so100003), (635, 709, so100005), (710, 799, so100002), (800, 859, so100001)]	859	0,7777
6	720	[(500, 559, so100004), (560, 634, so100003), (635, 709, so100005), (710, 799, so100002), (800, 859, so100001), (860, 919, so100006)]	919	4,2000
7	5040	[(500, 559, so100004), (560, 634, so100003), (635, 709, so100005), (710, 799, so100002), (800, 889, so100007), (890, 949, so100001), (950, 1009, so100006)]	1009	25,8649
8	40320	[(500, 559, so100004), (560, 649, so100007), (650, 709, so100008), (710, 799, so100002), (800, 874, so100003), (875, 934, so100001), (935, 994, so100006), (995, ..., ...)]	1069	66,8905
9	362880	[(480, 539, so100001), (540, 614, so100005), (620, 679, so100008), (680, 769, so100002), (770, 829, so100009), (830, 904, so100003), (905, 964, so100006), (965, ..., ...), (...)]	1114	299,3133
10	3628800	[(480, 539, so100001), (540, 629, so100007), (630, 689, so100008), (690, 779, so100002), (780, 839, so100009), (840, 899, so100010), (900, 959, so100006), (961, ..., ...), (...)]	1170	1982,8509
11	39916800	[(480, 539, so100001), (540, 629, so100007), (630, 689, so100008), (690, 779, so100002), (780, 839, so100009), (840, 899, so100006), (900, 974, so100011), (975, ..., ...), (...)]	1244	8140,0948



US6.3.3 - Heurísticas

As an Admin, I want to obtain a good schedule, not necessarily the better, in useful time to be adopted.

A heurística desenvolvida tem como objetivo a ordenação das cirurgias pela sua duração. Para isso, foi criado um predicado para somar a duração das fases (para ser usado ao incluir todo o staff) e também criado um predicado para ordenação.

Foram adicionados os seguintes predicados:

```
surgery_total_duration(OpCode, TotalDuration) :-  
    surgery_id(OpCode, OpType),  
    surgery(OpType, _, Second, _),  
    TotalDuration is Second.
```

```
sort_surgeries_by_duration(LOpCode, LOpCodeSorted) :-  
    % LOpDurations contem uma lista do tipo  
    % [Duration-OpCode, Duration-OpCode, Duration-OpCode]  
    findall(Duration-OpCode, (  
        % vai buscar um OpCode para cada elemento da lista  
        member(OpCode, LOpCode),  
        % vai buscar a duracao total da operacao  
        surgery_total_duration(OpCode, Duration)  
    ), LOpDurations),  
    % ordena uma lista de pares (chave-valor) pela chave, neste caso a duração  
    keysort(LOpDurations, LOpDurationsSortedPairs),  
    % retorna a lista dos códigos de operações, ordenado  
    pairs_values(LOpDurationsSortedPairs, LOpCodeSorted).
```

No `schedule_all_surgeries` foi também adicionado o seguinte predicado, antes do `availability_all_surgeries`:

```
% ordenar cirurgias por duração total  
sort_surgeries_by_duration(LOpCode, LOpCodeSorted),
```

N. of Surgeries	Optimal solution	Final Time for the last Surgery (minutes)	Final Time for the last Surgery Using Heuristic (minutes)	Time to generate the solution (s)	Time to generate the heuristic solution (s)	Solution with the Heuristic
3	[(480, 539, so100001), (540, 629, so100002), (630, 704, so100003)]	704	704	0,0119	0,000	[(480,539,so100001),(540,614,so100003),(615,704,so100002)]
4	[(480, 539, so100001), (540, 614, so100003), (615, 674, so100004), (675, 764, so100002)]	764	764	0,1567	0,000	[(480,539,so100001),(540,599,so100004),(600,674,so100003),(675,764,so100002)]
5	[(500, 559, so100004), (560, 634, so100003), (635, 709, so100005), (710, 799, so100002), (800, 859, so100001)]	859	1055	0,7777	0,000	[(480,539,so100001),(540,599,so100004),(600,674,so100003),(675,764,so100002),(981,1055,so100005)]
6	[(500, 559, so100004), (560, 634, so100003), (635, 709, so100005), (710, 799, so100002), (800, 859, so100001), (860, 919, so100006)]	919	1055	4,2000	0,000	[(480,539,so100001),(540,599,so100004),(600,659,so100006),(660,749,so100002),(791,865,so100003),(981,1055,so100005)]
7	[(500, 559, so100004), (560, 634, so100003), (635, 709, so100005), (710, 799, so100002), (800, 889, so100007), (890, 949, so100001), (950, 1009, so100006)]	1009	1145	25,8649	0,000	[(480,539,so100001),(540,599,so100004),(600,659,so100006),(660,749,so100002),(791,865,so100003),(981,1055,so100005),(1056,1145,so100007)]
8	[(500, 559, so100004), (560, 649, so100007), (650, 709, so100008), (710, 799, so100002), (800, 874, so100003), (875, 934, so100001), (935, 994, so100006), (995, ..., ...)]	1069	1235	66,8905	0,000	[(480,539,so100001),(540,599,so100004),(600,659,so100006),(660,719,so100008),(791,865,so100003),(981,1055,so100005),(1056,1145,so100002),(1146,1235,so100007)]
9	[(480, 539, so100001), (540, 614, so100005), (620, 679, so100008), (680, 769, so100002), (770, 829, so100009), (830, 904, so100003), (905, 964, so100006), (965, ..., ...), (... , ...)]	1114	1235	299,3133	0,000	[(480,539,so100001),(540,599,so100004),(600,659,so100006),(660,719,so100008),(720,779,so100009),(791,865,so100003),(981,1055,so100005),(1056,1145,so100002),(1146,1235,so100007)]
10	[(480, 539, so100001), (540, 629, so100007), (630, 689, so100008), (690, 779, so100002), (780, 839, so100009), (840, 899, so100010), (900, 959, so100006), (961, ..., ...), (... , ...) ...]	1170	1310	1982,8509	0,000	[(480,539,so100001),(540,599,so100004),(600,659,so100006),(660,719,so100008),(720,779,so100009),(791,850,so100010),(981,1055,so100003),(1056,1130,so100005),(1131,1220,so100002),(1221,1310,so100007)]
11	[(480, 539, so100001), (540, 629, so100007), (630, 689, so100008), (690, 779, so100002), (780, 839, so100009), (840, 899, so100006), (900, 974, so100011), (975, ..., ...), (... , ...) ...]	1244	1310	8140,0948	0,000	[(480,539,so100001),(540,599,so100004),(600,659,so100006),(660,719,so100008),(720,779,so100009),(791,850,so100010),(851,925,so100011),(981,1055,so100003),(1056,1130,so100005),(1131,1220,so100002),(1221,1310,so100007)]

US6.3.1 – Incluir todo o staff

As an Admin, I want to obtain the better scheduling of a set of operations (surgeries) in a certain operation room in a specific day.

Foram adicionados horários para os novos staff:

timetable(a001,20241028,(550,1200)).

timetable(a002,20241028,(600,1200)).

timetable(a003,20241028,(550,1200)).

timetable(a004,20241028,(600,1200)).

timetable(a005,20241028,(550,1200)).

timetable(a006,20241028,(600,1200)).

timetable(c001,20241028,(480,1200)).

timetable(c002,20241028,(480,1200)).

timetable(c003,20241028,(480,1200)).

E depois atribuídos a operações:

assignment_anaesthetist(so100001,a001).

assignment_anaesthetist(so100001,a002).

assignment_anaesthetist(so100002,a003).

assignment_anaesthetist(so100002,a004).

assignment_anaesthetist(so100003,a005).

assignment_anaesthetist(so100003,a006).

assignment_cleaner(so100001,c001).

assignment_cleaner(so100002,c002).

assignment_cleaner(so100003,c003).

Abaixo está representado o predicado `availability_all_surgeries` em que as alterações estão sublinhadas:

```
availability_all_surgeries([],_,_).
```

```
availability_all_surgeries([OpCode|LOpCode],Room,Day):-
```

```
    % busca o tipo de cirurgia a partir do código
```

```
    surgery_id(OpCode,OpType),
```

```
    % busca o tempo da operacao atraves do tipo
```

```
    surgery(OpType,TAnesthesia,TSurgery,TCleaning),
```

```
    OperationDuration is TAnesthesia+TSurgery+TCleaning.
```

```
    % retorna uma lista de possibilidades de horarios (inicio,fim) e uma lista dos médicos
```

```
    availability_operation(OpCode,Room,Day,LPossibilities,LDoctors,LAnaesthetists,LCleaners),
```

```
    % da lista de possibilidades retorna a primeira
```

```
    schedule_first_interval(OperationDuration,LPossibilities,(TinS,_)),
```

```
    % calcular início e fim do horário para cada tipo de staff
```

```
    being_end_phases(TinS,TAnesthesia,TSurgery,TCleaning,AnesthetistSlot,DoctorSlot,CleanerSlot,OpCode).
```

```
    % atribuir o horario a cada um do tipo de staff
```

```
    insert_agenda_staff(DoctorSlot,Day,LDoctors).
```

```
    insert_agenda_staff(AnesthetistSlot,Day,LAnaesthetists).
```

```
    insert_agenda_staff(CleanerSlot,Day,LCleaners).
```

```
    % retira da agenda da sala as marcacoes
```

```
    retract(agenda_operation_room1(Room,Day,Agenda)),
```

```
    % insere nessa lista a nova marcacao
```

```
    TfinS is TinS+OperationDuration-1.
```

```
    insert_agenda((TinS,TfinS,OpCode),Agenda,Agenda1),
```

```
    % reinsere a lista na agenda da sala
```

```
    assertz(agenda_operation_room1(Room,Day,Agenda1)),
```

```
    % recursivo para o resto das operacoes
```

```
    availability_all_surgeries(LOpCode,Room,Day).
```

Predicado auxiliar ao anterior:

% calcular início e fim do horario para cada tipo de staff

being_end_phases(TStart,TAnesthesia,TSurgery,TCleaning,AnesthetistSlot,DoctorSlot,CleanerSlot,OpCode):-

AnesthetistSlotFim is TStart+TAnesthesia+TSurgery-1,

AnesthetistSlot=(TStart,AnesthetistSlotFim,OpCode),

DoctorSlotIni is TStart+TAnesthesia,

DoctorSlotFim is TStart+TAnesthesia+TSurgery-1,

DoctorSlot=(DoctorSlotIni,DoctorSlotFim,OpCode),

CleanerSlotIni is TStart+TAnesthesia+TSurgery,

CleanerSlotFim is TStart+TAnesthesia+TSurgery+TCleaning-1,

CleanerSlot=(CleanerSlotIni,CleanerSlotFim,OpCode).

Abaixo está representado o predicado `availability_operation` em que as alterações estão sublinhadas:

`availability_operation(OpCode,Room,Day,LPossibilities,LDoctors,LAnaesthetists,LCleaners):-`

`% busca o tipo de cirurgia a partir do codigo`

`surgery_id(OpCode,OpType),`

`% busca o tempo da operacao atraves do tipo`

`surgery(OpType,TAnesthesia,TSurgery,TCleaning),`

`% duracao total somando as 3 fases`

`OperationDuration is TAnesthesia+TSurgery+TCleaning.`

`% constroi uma lista dos cirurgioes necessarios para a operacao`

`findall(Doctor,assignment_surgery(OpCode,Doctor),LDoctors),`

`% constroi uma lista dos anestesista necessarios para a operacao`

`findall(Anaesthetist,assignment_anaesthetist(OpCode,Anaesthetist),LAnaesthetists).`

`% constroi uma lista dos limpadores necessarios para a operacao`

`findall(Cleaner,assignment_cleaner(OpCode,Cleaner),LCleaners).`

`% juntar as 3 listas`

`append([LDoctors,LAnaesthetists,LCleaners],LStaff).`

`% interseta as agendas do staff para o dia -> LA`

`intersect_all_agendas(LStaff,Day,LA),`

`% busca as marcacoes para a sala -> LAgenda`

`agenda_operation_room1(Room,Day,LAgenda),`

`% calcula os tempos livres para a sala -> LFAgRoom`

`free_agenda0(LAgenda,LFAgRoom),`

`% interseta a agenda livre da sala de operacao com as agendas livres dos medicos`

`intersect_2_agendas(LA,LFAgRoom,LIntAgDoctorsRoom),`

`% remove da lista (de intersecao de agenda dos medicos com a sala) todos os slots que sejam menores que o tempo da operacao (TSurgery)`

`remove_unf_intervals(OperationDuration,LIntAgDoctorsRoom,LPossibilities).`

Conclusão

Este projeto foi uma boa oportunidade para pôr em prática os conhecimentos adquiridos nas aulas sobre algoritmos e otimização, aplicando-os a situações mais concretas como é o caso deste projeto de agendamento de cirurgias.

Fomos capazes de incluir as diferentes categorias de staff e as várias fases da cirurgia que culminou num agendamento que contempla todo o staff respeitando as restrições.

Concluímos com sucesso a criação de uma heurística rápida, apesar de não ser a solução ótima, a heurística implementada permite criar bons horários num tempo razoável.